

Convolutional neural networks, part 2

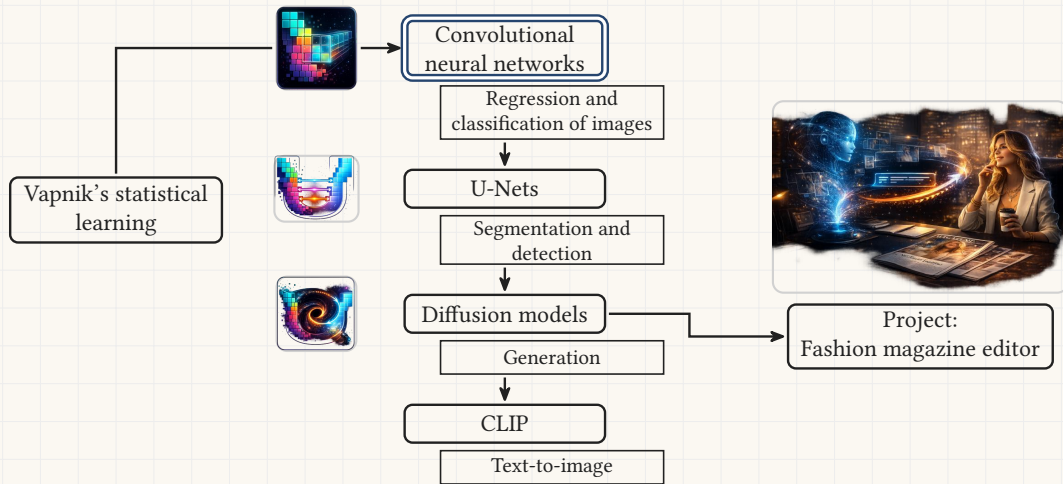
Weekend 3, Friday morning

Carlos Cotrini

CAS Building ML/AI Applications, ETH Zurich

Friday 2 October 2026

Agenda

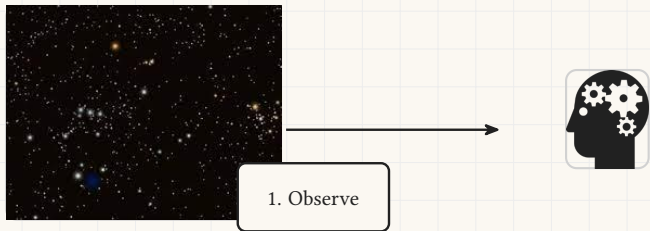


1

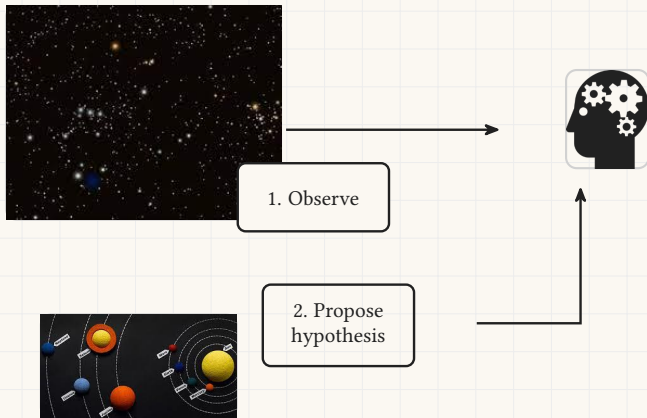
Section 1

Motivation

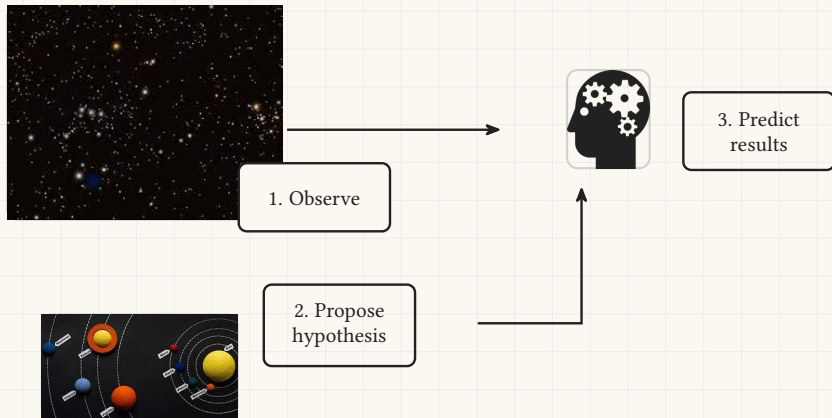
How humans create knowledge



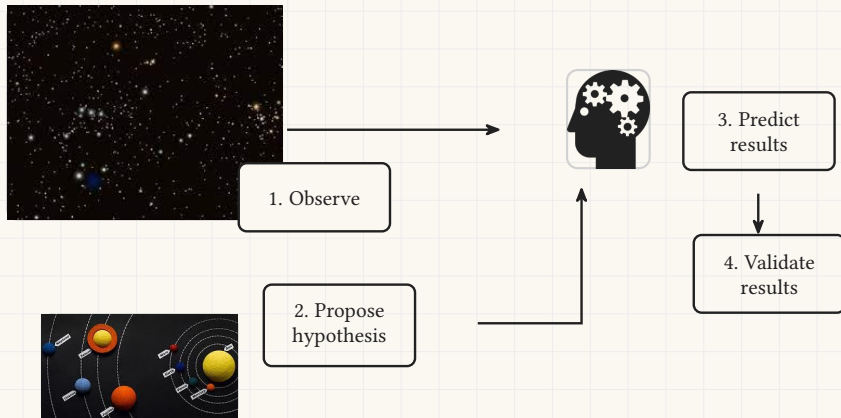
How humans create knowledge



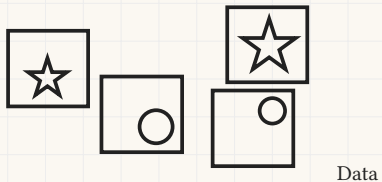
How humans create knowledge



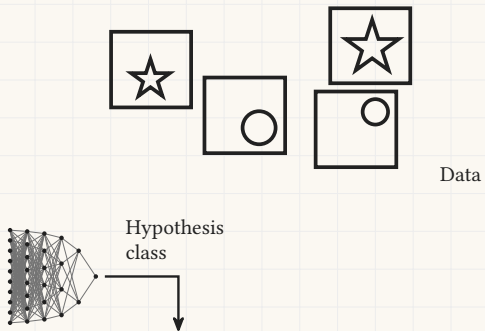
How humans create knowledge



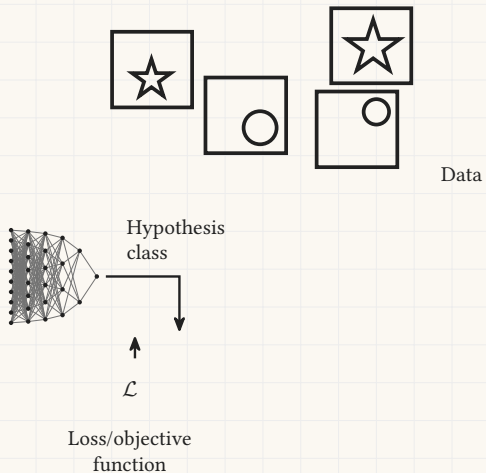
Statistical learning in data science



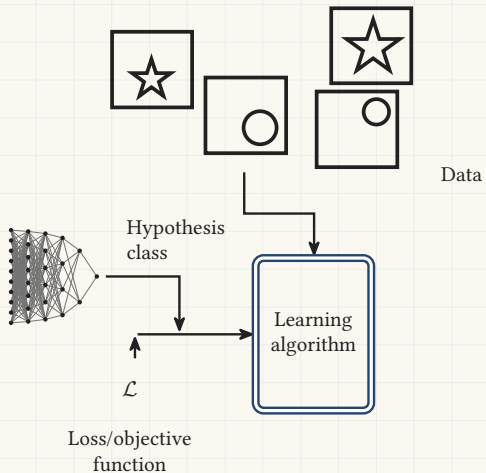
Statistical learning in data science



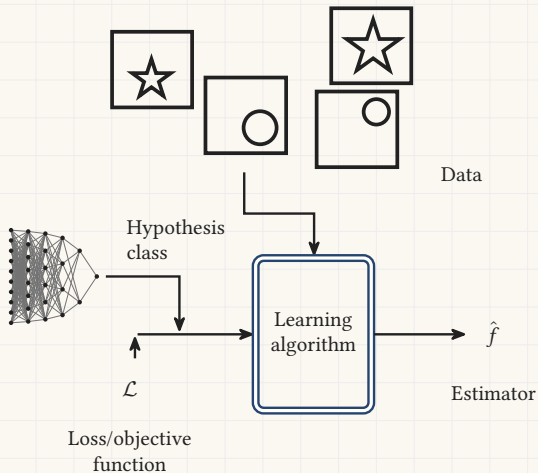
Statistical learning in data science



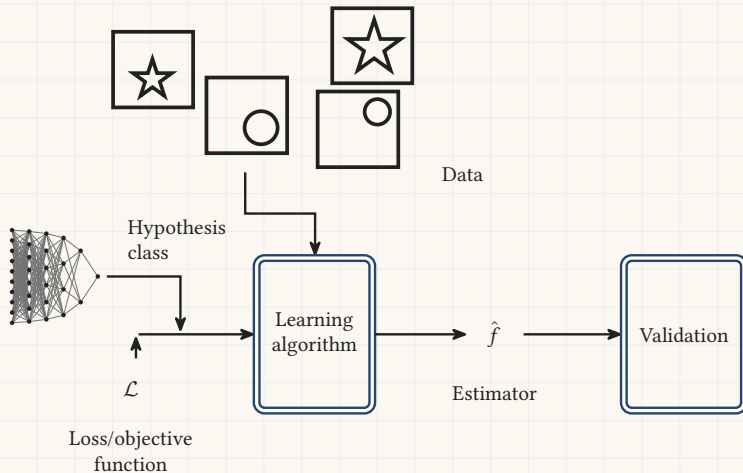
Statistical learning in data science



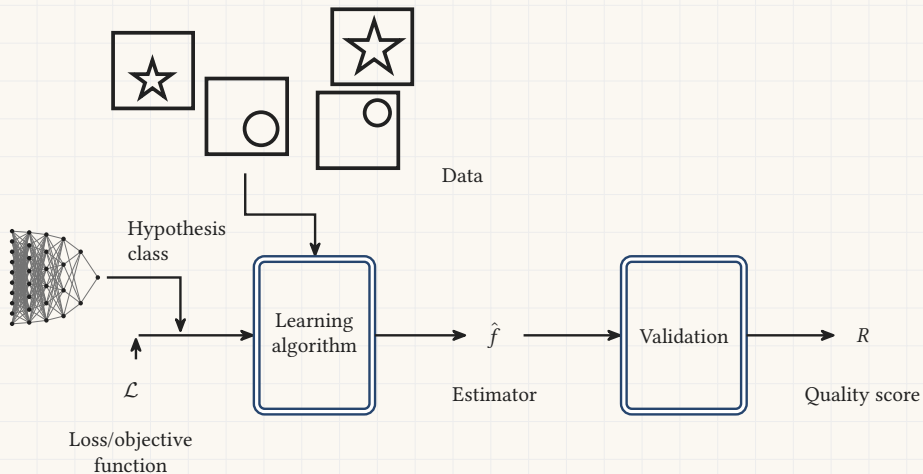
Statistical learning in data science



Statistical learning in data science



Statistical learning in data science



Classifying stars and planets



Star



Star



Star

Classifying stars and planets



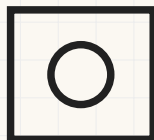
Star



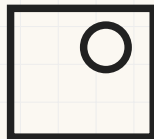
Star



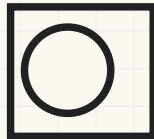
Star



Planet



Planet



Planet

2

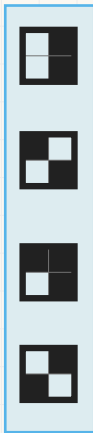
Section 2

Convolutional neural networks

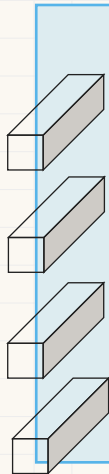
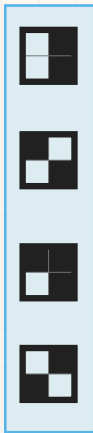
Convolutional neural networks



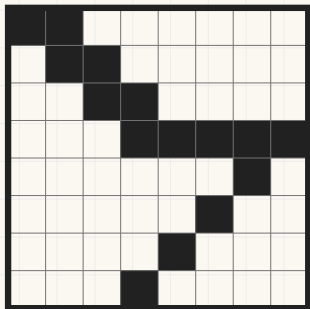
Convolutional neural networks



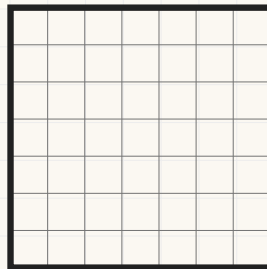
Convolutional neural networks



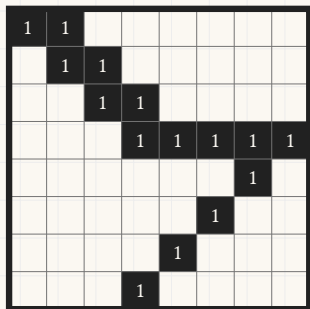
Convolutional neural networks



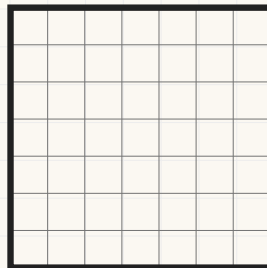
Convolutional filter



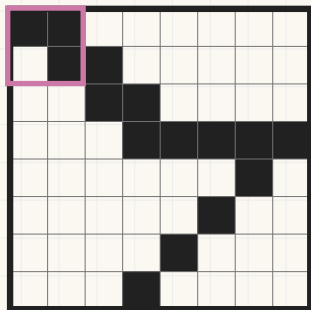
Convolutional neural networks



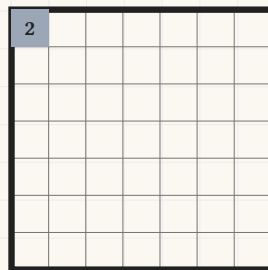
Convolutional filter



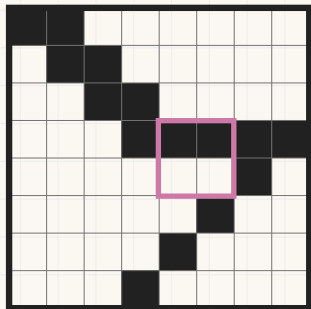
Convolutional neural networks



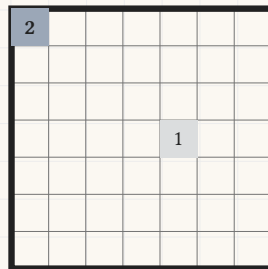
Convolutional filter



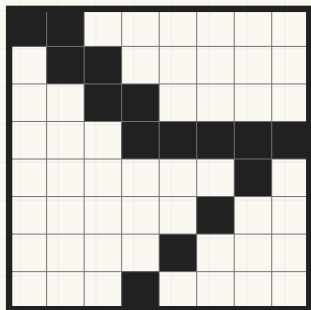
Convolutional neural networks



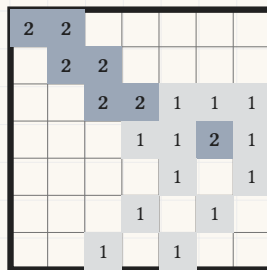
Convolutional filter



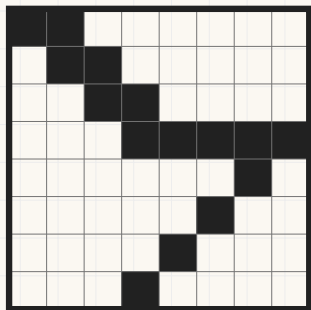
Convolutional neural networks



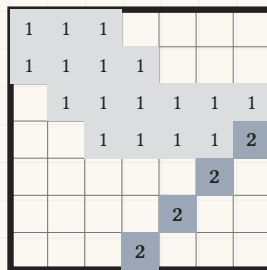
Convolutional filter



Convolutional neural networks



Convolutional filter



Filters

1	3	5
1	2	4
6	3	2

Input

1	2
-5	0

Convolutional filter

ReLU

Output

Filters

1	3	5
1	2	4
6	3	2

Input

1	2
-5	0

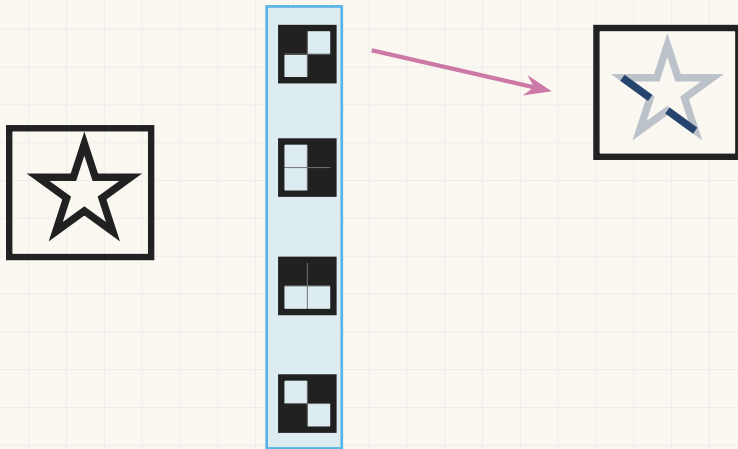
Convolutional filter

ReLU

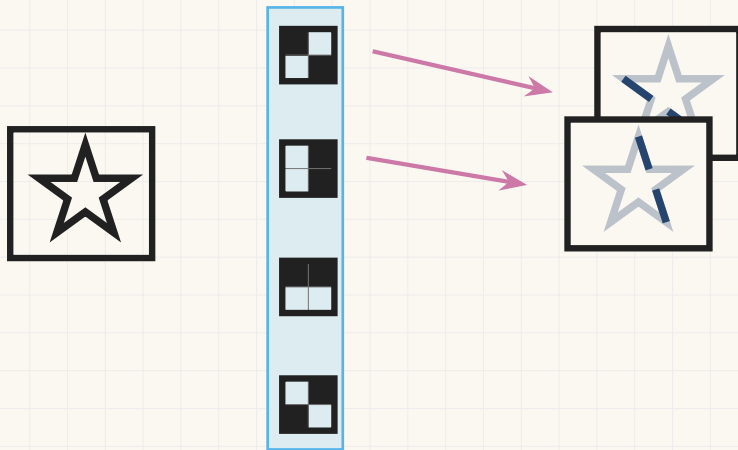
2	3
0	0

Output

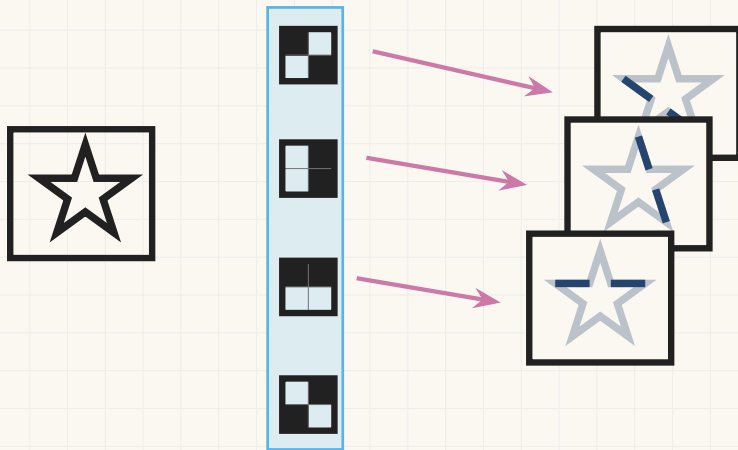
Convolutional neural networks



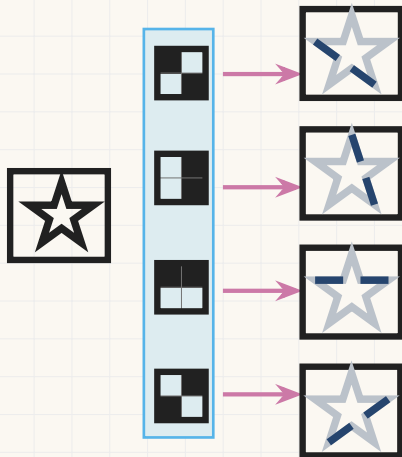
Convolutional neural networks



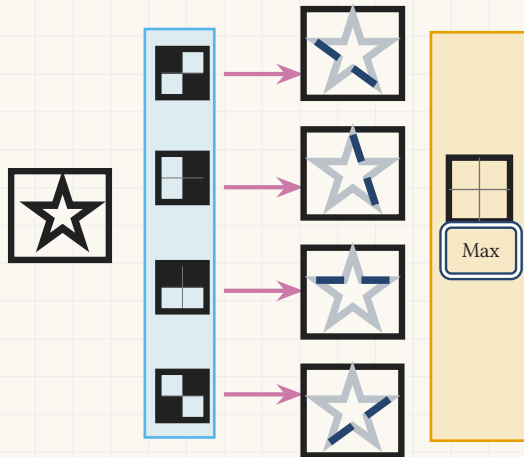
Convolutional neural networks



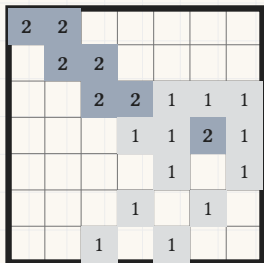
Convolutional neural networks



Convolutional neural networks



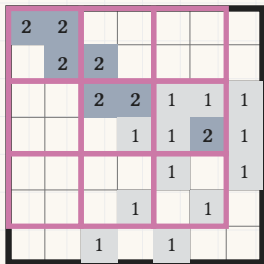
Convolutional neural networks



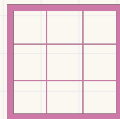
Max



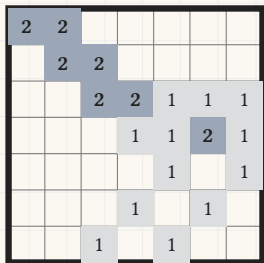
Convolutional neural networks



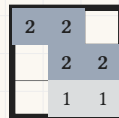
Max



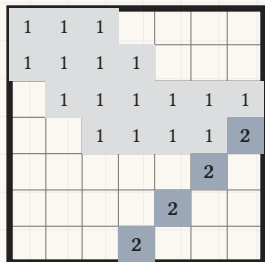
Convolutional neural networks



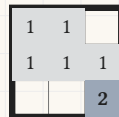
Max



Convolutional neural networks



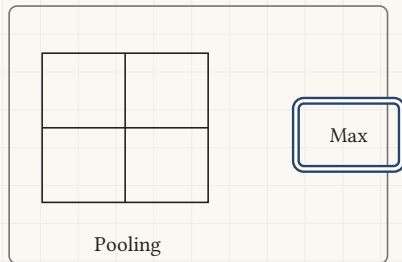
Max



Pooling

1	0	5	3	1
4	0	0	2	2
6	1	0	1	0
0	0	0	2	4
3	0	1	2	1

Input

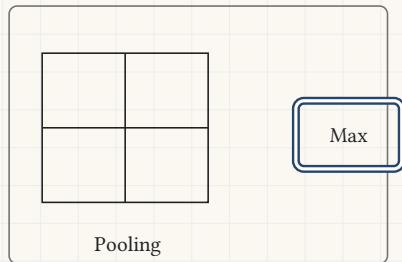


Output

Pooling

1	0	5	3	1
4	0	0	2	2
6	1	0	1	0
0	0	0	2	4
3	0	1	2	1

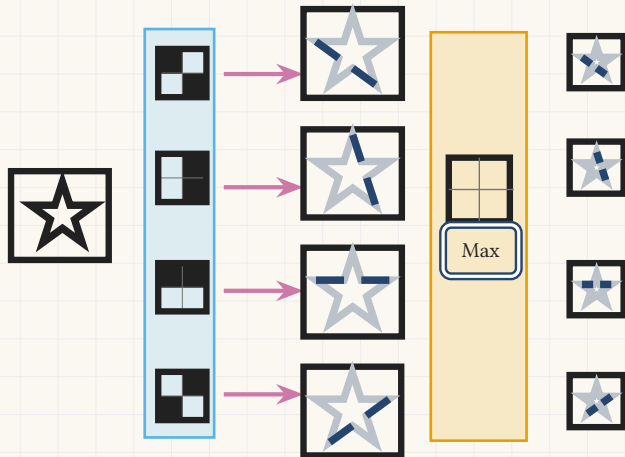
Input



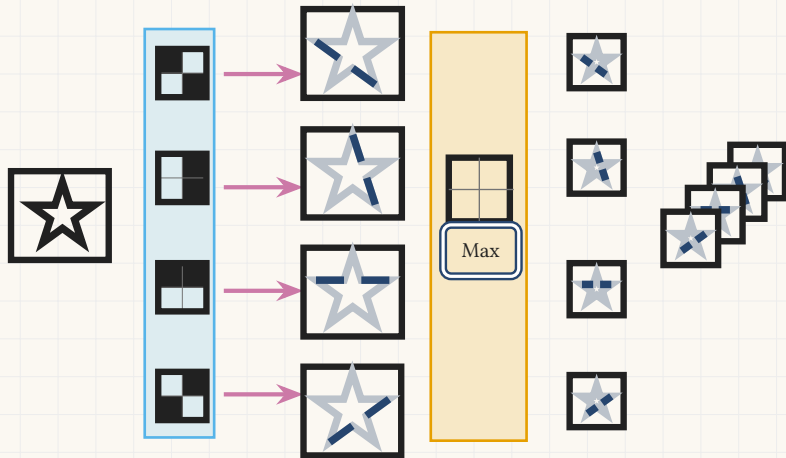
4	5
6	2

Output

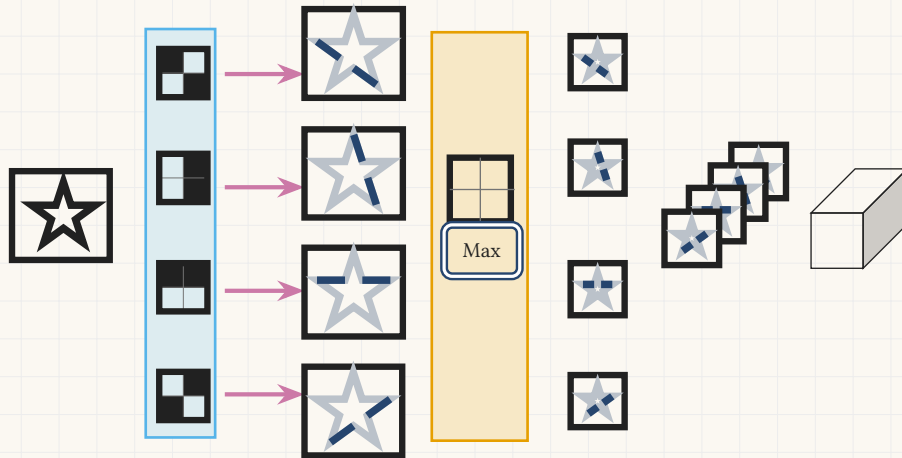
Convolutional neural networks



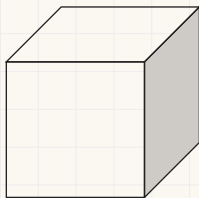
Convolutional neural networks



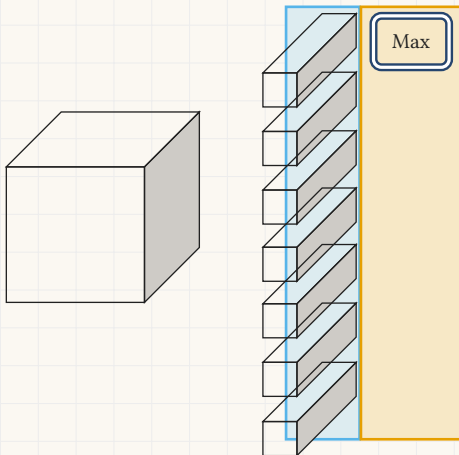
Convolutional neural networks



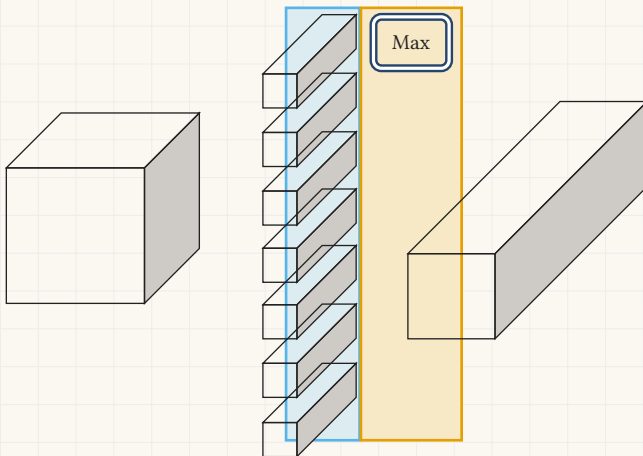
Convolutional neural networks



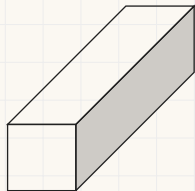
Convolutional neural networks



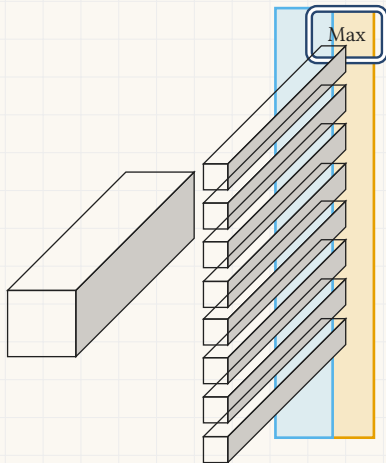
Convolutional neural networks



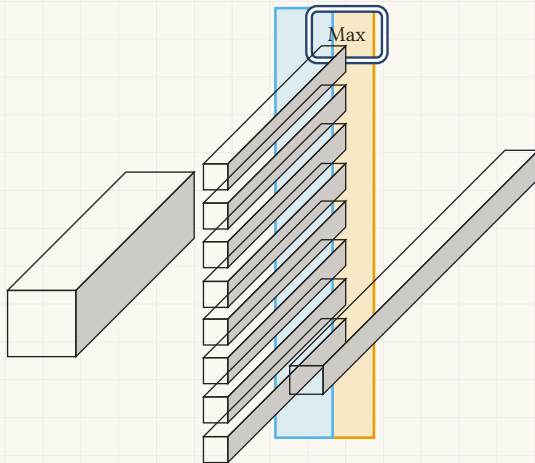
Convolutional neural networks



Convolutional neural networks



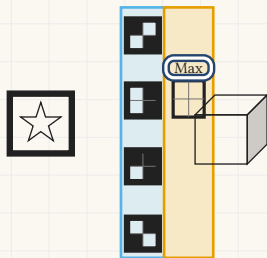
Convolutional neural networks



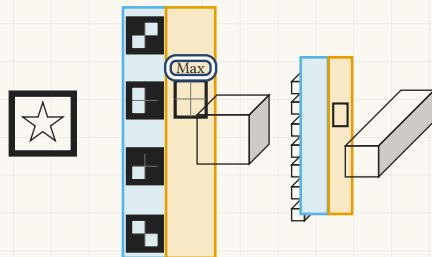
Convolutional neural networks



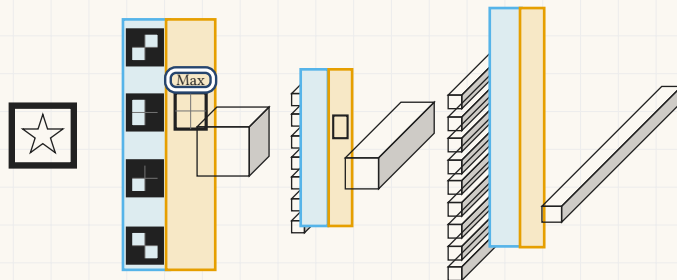
Convolutional neural networks



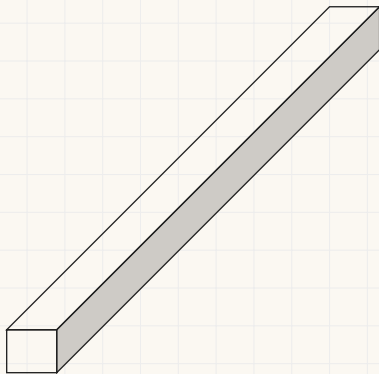
Convolutional neural networks



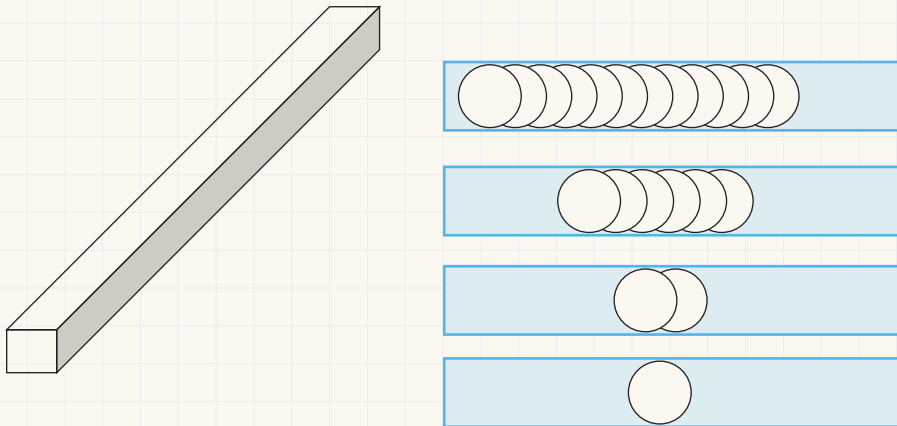
Convolutional neural networks



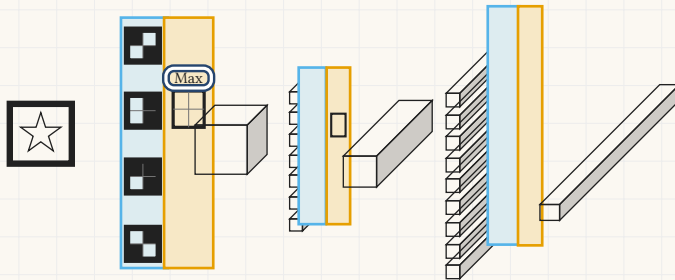
Convolutional neural networks



Convolutional neural networks



Convolutional neural networks



Convolutional neural networks

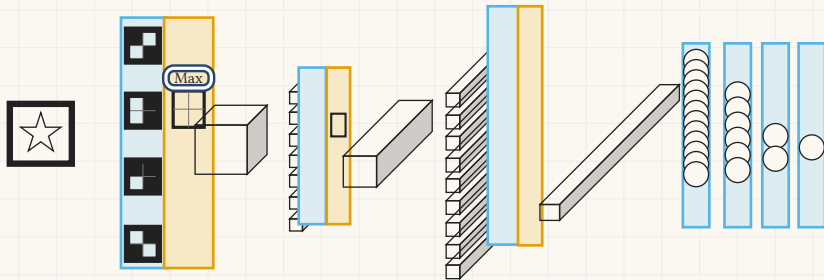


Image recognition

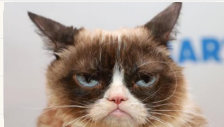
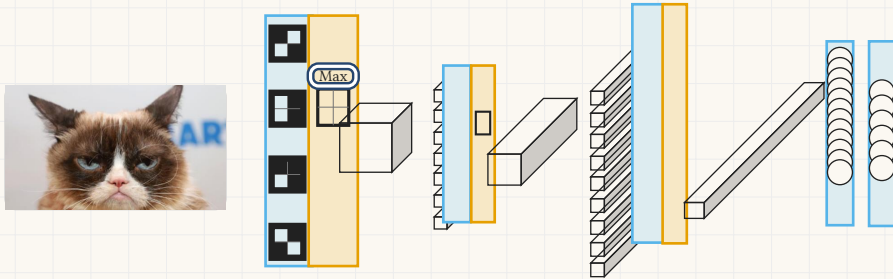
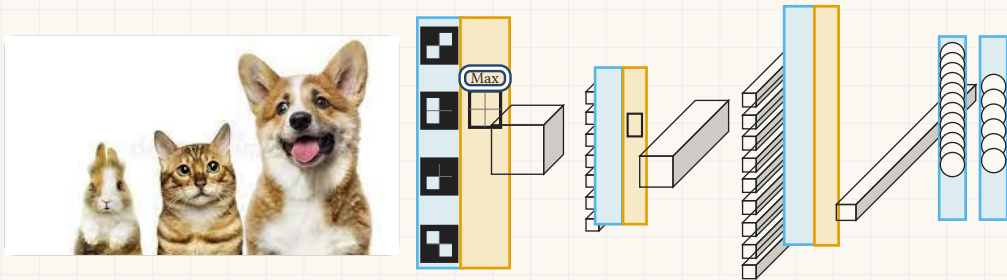


Image recognition



Softmax



Softmax



Softmax

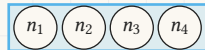
Binary classification: one of two classes

- One neuron at the end, sigmoid activation
- A value between 0 and 1: the probability of that one class

Softmax

Binary classification: one of two classes

- One neuron at the end, sigmoid activation
- A value between 0 and 1: the probability of that one class



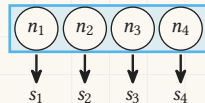
Multi-class classification: one of K classes

- K neurons $n_1, n_2, \dots, n_K$ at the end, one score each
- A softmax turns the scores into probabilities
 $p_1, p_2, \dots, p_K$
- Each p_i : the probability of class i

Softmax

Binary classification: one of two classes

- One neuron at the end, sigmoid activation
- A value between 0 and 1: the probability of that one class



Multi-class classification: one of K classes

- K neurons $n_1, n_2, \dots, n_K$ at the end, one score each
- A softmax turns the scores into probabilities
 $p_1, p_2, \dots, p_K$
- Each p_i : the probability of class i

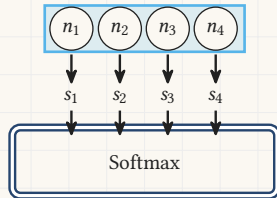
Softmax

Binary classification: one of two classes

- One neuron at the end, sigmoid activation
- A value between 0 and 1: the probability of that one class

Multi-class classification: one of K classes

- K neurons $n_1, n_2, \dots, n_K$ at the end, one score each
- A softmax turns the scores into probabilities
 $p_1, p_2, \dots, p_K$
- Each p_i : the probability of class i



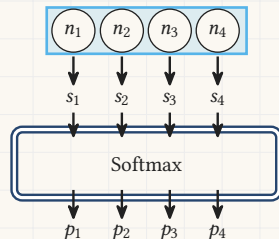
Softmax

Binary classification: one of two classes

- One neuron at the end, sigmoid activation
- A value between 0 and 1: the probability of that one class

Multi-class classification: one of K classes

- K neurons $n_1, n_2, \dots, n_K$ at the end, one score each
- A softmax turns the scores into probabilities $p_1, p_2, \dots, p_K$
- Each p_i : the probability of class i

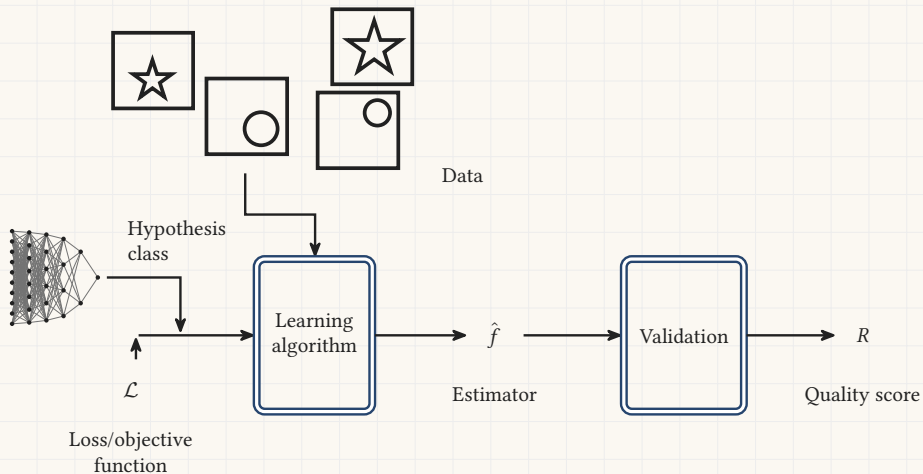


3

Section 3

Training and validation

Statistical learning in data science



Loss/objective function

1. A function that takes as input:
 - A prediction output by the neural network for an example
 - The label of that example

Loss/objective function

1. A function that takes as input:
 - A prediction output by the neural network for an example
 - The label of that example
2. Its output is a real number measuring how “bad” the prediction is with respect to the example

Loss/objective function

1. A function that takes as input:
 - A prediction output by the neural network for an example
 - The label of that example
2. Its output is a real number measuring how “bad” the prediction is with respect to the example
3. A popular loss function for classification: the ***cross-entropy loss function***

Cross-entropy loss function

1. Input: the network's prediction for a particular example, as a probability distribution $(p_1, \dots, p_K)$ over the K different classes

Cross-entropy loss function

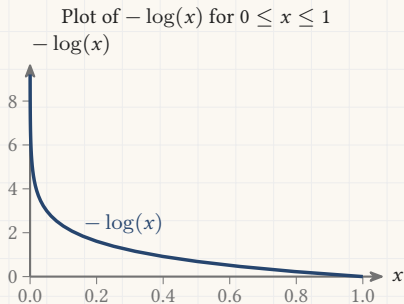
1. Input: the network's prediction for a particular example, as a probability distribution $(p_1, \dots, p_K)$ over the K different classes
2. Input: the actual label $\ell \leq K$ of the example, called **the ground truth**

Cross-entropy loss function

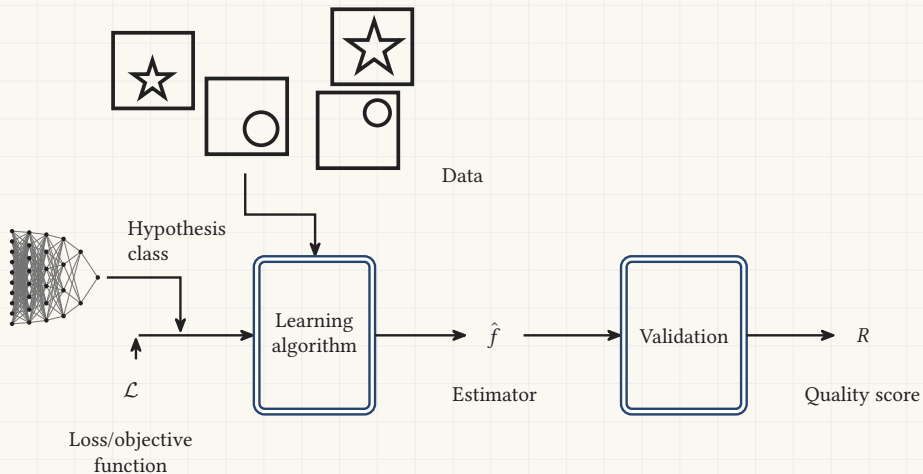
1. Input: the network's prediction for a particular example, as a probability distribution $(p_1, \dots, p_K)$ over the K different classes
2. Input: the actual label $\ell \leq K$ of the example, called **the ground truth**
3. Output: $-\log p_\ell$

Cross-entropy loss function

1. Input: the network's prediction for a particular example, as a probability distribution $(p_1, \dots, p_K)$ over the K different classes
2. Input: the actual label $\ell \leq K$ of the example, called **the ground truth**
3. Output: $-\log p_\ell$



Statistical learning in data science



Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

1. Make an initial guess for the parameters of your estimator

Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

1. Make an initial guess for the parameters of your estimator
2. Evaluate the loss function on your dataset

Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

1. Make an initial guess for the parameters of your estimator
2. Evaluate the loss function on your dataset
3. Compute the gradient of the loss

Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

1. Make an initial guess for the parameters of your estimator
2. Evaluate the loss function on your dataset
3. Compute the gradient of the loss
4. Correct the parameters of the estimator according to the gradient

Training: gradient descent



Most ML algorithms use gradient descent to find a good estimator for a given dataset.

1. Make an initial guess for the parameters of your estimator
2. Evaluate the loss function on your dataset
3. Compute the gradient of the loss
4. Correct the parameters of the estimator according to the gradient
5. Repeat from step 2 until the loss stops decreasing

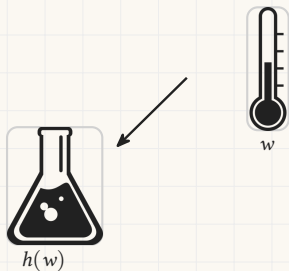
Reactor stabilization game



w

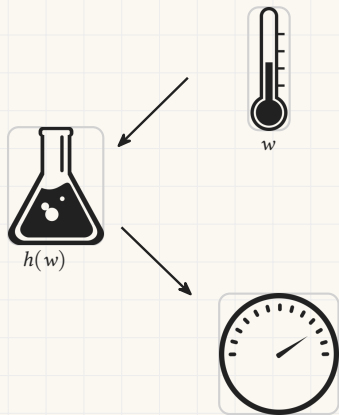
- A highly sensitive chemical reaction in a research lab
- Kept at a precise temperature w^* : the “root” of a function h tied to the reaction’s equilibrium

Reactor stabilization game



- A highly sensitive chemical reaction in a research lab
- Kept at a precise temperature w^* : the “root” of a function h tied to the reaction’s equilibrium
- We have no direct access to h

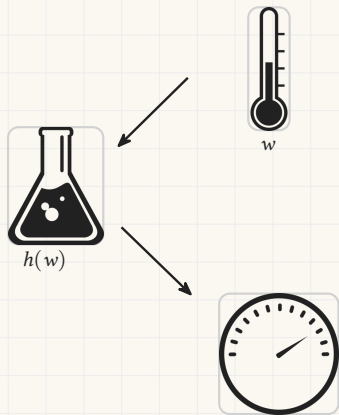
Reactor stabilization game



- A highly sensitive chemical reaction in a research lab
- Kept at a precise temperature w^* : the “root” of a function h tied to the reaction’s equilibrium
- We have no direct access to h
- A noisy device approximately measures the pH level

$$\Delta(w) = h(w) + \epsilon, \text{ with } \mathbb{E}[\epsilon] = 0$$

Reactor stabilization game



- A highly sensitive chemical reaction in a research lab
- Kept at a precise temperature w^* : the “root” of a function h tied to the reaction’s equilibrium
- We have no direct access to h
- A noisy device approximately measures the pH level

Link to the game:

https://colab.research.google.com/drive/1HCo-C4bSCPUScAiX7BiGcQOdM_FO-Zym?usp=sharing



$$\Delta(w) = h(w) + \epsilon, \text{ with } \mathbb{E}[\epsilon] = 0$$

Connection to gradient descent

- We learned to reach the pH level of 7 by noting: lowering the temperature lowers the pH level, increasing it increases the pH level

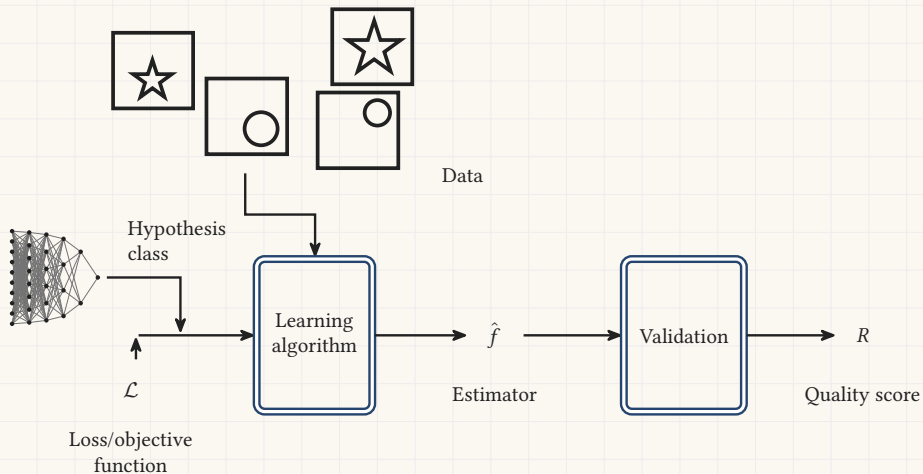
Connection to gradient descent

- We learned to reach the pH level of 7 by noting: lowering the temperature lowers the pH level, increasing it increases the pH level
- For an analytical function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ we can compute its gradient $\nabla \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$
 - Minus the gradient, $-\nabla \mathcal{L}$, points in the direction that decreases $\mathcal{L}$ the fastest

Connection to gradient descent

- We learned to reach the pH level of 7 by noting: lowering the temperature lowers the pH level, increasing it increases the pH level
- For an analytical function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ we can compute its gradient $\nabla \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$
 - Minus the gradient, $-\nabla \mathcal{L}$, points in the direction that decreases $\mathcal{L}$ the fastest
- The gradient shows how to tweak the parameters of a neural network to decrease the loss function the fastest

Statistical learning in data science



Validation

- Once you have computed an estimator (a neural network) with gradient descent, validate it on a separate dataset

Validation

- Once you have computed an estimator (a neural network) with gradient descent, validate it on a separate dataset
- Why? Too complex neural networks may risk overfitting your dataset